VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

MÃ QUỐC CƯỜNG – 522I0001
NGUYỄN CAO PHI – 521V0006

A Comparative Analysis of FPMax and GenMax for Mining Frequent Maximal Itemsets from Uncertain Transaction Databases

Information Technology Project – 504091

HO CHI MINH CITY, YEAR 2026

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY
888

MÃ QUỐC CƯỜNG – 522I0001
NGUYỄN CAO PHI – 521V0006

A Comparative Analysis of FPMax and GenMax for Mining Frequent Maximal Itemsets from Uncertain Transaction Databases

Information Technology Project – 504091

Advised by
Prof. Nguyễn Chí Thiện

HO CHI MINH CITY, YEAR 2026

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Professor Nguyễn Chí Thiện for his patient guidance, thoughtful supervision, and continuous encouragement throughout this project. His expertise in data mining and algorithm design provided invaluable direction at every stage of our work.

We also thank our classmates for their helpful discussions and feedback, and Ton Duc Thang University for providing the academic environment and resources that made this project possible.

Ho Chi Minh City, day month year 2026

Authors

(Signature and full name)

DECLARATION OF AUTHORSHIP

I hereby declare that this thesis was carried out by my team under the guidance and supervision of Prof Nguyễn Chí Thiện; and that the work and the results contained in it are original and have not been submitted anywhere for any previous purposes. The data and figures presented in this thesis are for analysis, comments, and evaluations from various resources by my own work and have been duly acknowledged in the reference part.

In addition, other comments, reviews and data used by other authors, and organizations have been acknowledged, and explicitly cited.

I will take full responsibility for any fraud detected in my thesis. Ton Duc Thang University is unrelated to any copyright infringement caused on my work (if any).

Ho Chi Minh City, day month year
Author

[Signature and full name]

ABSTRACT

Frequent itemset mining is a fundamental task in data mining. Classical algorithms assume binary transaction data where items are either present or absent. However, real-world databases often contain **uncertainty**: a barcode scanner might be only 80% confident that it read a product correctly, a sensor might report a reading with known error margins, or data merged from multiple sources might contain conflicting entries. In such databases, each item is associated with an **existential probability** rather than a simple yes/no flag.

This report addresses the problem of mining **frequent maximal itemsets** from uncertain transaction databases using two complementary support models. We implement and evaluate four algorithms:

Under the **expected support model**: **UGenMax** adapts GenMax (Gouda and Zaki, 2005) using bottom-up weighted tidset intersections, and **UFPMax** adapts FPMax (Grahne and Zhu, 2003) using top-down conditional database projection. Both employ branch-and-bound search with anti-monotonicity pruning, progressive focusing, and dynamic Top-K threshold raising.

Under the **probabilistic support model**: **TODIS-MAX** (based on Sun et al., 2010) provides exact probabilistic frequency computation via Dynamic Programming, and **APFI-MAX** (based on Chen et al., 2020) provides faster approximate estimation via the Central Limit Theorem.

All algorithms are implemented in pure Java SE with no external dependencies. Experiments on three benchmark datasets (mushroom, retail, accidents) at three uncertainty levels show that: (1) UGenMax and UFPMax produce identical results, with UGenMax being 15–50% faster; (2) TODIS-MAX and APFI-MAX produce matching results, with APFI-MAX being 3–8× faster; (3) all algorithms scale to 340K transactions.

Contents

ACKNOWLEDGMENT **1**

DECLARATION OF AUTHORSHIP **2**

ABSTRACT **3**

TABLE OF CONTENTS **6**

IMAGE TABLE **6**

List of Abbreviations **7**

1 INTRODUCTION **8**

1.1 What is Frequent Itemset Mining? 8
1.2 The Challenge of Uncertainty 8
1.3 Why Maximal Itemsets? 8
1.4 Project Objectives 9
1.5 Report Structure 9

2 LITERATURE REVIEW **10**

2.1 Frequent Itemset Mining in Deterministic Databases 10
2.2 Mining Maximal Frequent Itemsets 10
2.3 Uncertain Database Mining 10
2.4 Probabilistic Support Mining 10
2.5 Gap and Our Contribution 11

3 PROBLEM STATEMENT AND RELATED DEFINITIONS **12**

3.1 Uncertain Transaction Database 12
3.2 Expected Support 12
3.3 Anti-Monotonicity Property 13
3.4 Frequent and Maximal Frequent Itemsets 13
3.5 Probabilistic Support 13
3.6 Problem Statement 14

4 METHODS **15**
4.1 Branch-and-Bound Framework 15
4.1.1 Pruning 1: Anti-Monotonicity Bound 15
4.1.2 Pruning 2: Progressive Focusing 15
4.1.3 Pruning 3: Top-K Dynamic Threshold Raising 15
4.2 Algorithm 1: UGenMax 16
4.2.1 Data Structure: Weighted Tidsets 16
4.2.2 Enumeration Strategy 16
4.2.3 Pseudocode 17
4.2.4 Complexity Analysis 17
4.3 Algorithm 2: UFPMax 18
4.3.1 Data Structure: Projected Transactions 18
4.3.2 Conditional Database Construction 18
4.3.3 Pseudocode 18
4.3.4 Complexity Analysis 19
4.4 Comparison of UGenMax and UFPMax 19
4.5 Algorithm 3: TODIS-MAX 20
4.5.1 Probabilistic Support Computation 20
4.5.2 Algorithm Overview 20
4.5.3 Pseudocode 21
4.5.4 Complexity Analysis 21
4.6 Algorithm 4: APFI-MAX 21
4.6.1 CLT Approximation 21
4.6.2 Expectation Bounds 22
4.6.3 Algorithm Overview 22
4.6.4 Pseudocode 23
4.6.5 Complexity Analysis 23
4.7 Comparison of All Four Algorithms 23
4.8 Top-K Seeding 23
4.9 Maximality Checking 24

5 EXPERIMENT DESIGN **25**
5.1 Implementation Details 25
5.2 Datasets 25
5.3 Configurations 26
5.4 Correctness Validation 26
5.5 Performance Measurement 26

6 EXPERIMENT RESULTS **27**

6.1 Correctness Verification 27
6.2 Expected Support: Effect of Minimum Support Threshold 27
6.3 Scalability: Accidents Dataset 28
6.4 Effect of Uncertainty Level 28
6.5 Probabilistic Support: TODIS-MAX vs. APFI-MAX 29
6.6 Cross-Model Comparison 30
6.7 Summary of Findings 30

7 CONCLUSIONS 31
7.1 Summary 31
7.2 Key Contributions 31
7.3 Limitations 32
7.4 Future Work 32

List of Figures

LIST OF ABBREVIATIONS

Abbreviation	Definition
FIM	Frequent Itemset Mining
MFI	Maximal Frequent Itemset
DFS	Depth-First Search
expSup	Expected Support
minsup	Minimum Expected Support Threshold
WT	Weighted Tidset
DB	Database

CHAPTER 1
INTRODUCTION

1.1 What is Frequent Itemset Mining?

Imagine a supermarket analyzing thousands of customer receipts to discover which products are frequently bought together. If many customers buy bread and butter in the same trip, the pair {bread, butter} is a **frequent itemset**. This information is valuable for product placement, cross-selling, and inventory management.

More formally, given a database of transactions (where each transaction is a set of items) and a minimum support threshold, the goal of frequent itemset mining is to find all item groups that appear in at least that many transactions. This problem was first formalized by Agrawal, Imielinski, and Swami [1] and has since become one of the most studied problems in data mining, with applications in bioinformatics, text mining, web usage analysis, network security, and recommendation systems.

1.2 The Challenge of Uncertainty

Classical algorithms assume perfect data: an item is either in a transaction or it is not. In practice, data is often imperfect. Consider these real-world scenarios:

- **Sensor networks.** A temperature sensor reports values with a known error margin. When the system logs that a sensor detected item A, it might only be 85% confident in that reading.
- **Data integration.** When merging customer records from two different databases, some matches are ambiguous. A merged record might indicate that a customer purchased item B, but with only 70% confidence due to name/address mismatches.
- **Privacy preservation.** To protect user privacy, a system deliberately adds random noise to transaction data. Each item's presence is now probabilistic rather than certain.

In all these cases, the result is an **uncertain transaction database** where each item in each transaction carries a probability between 0 and 1. The challenge is: how do we mine meaningful patterns from data that we are not entirely sure about?

1.3 Why Maximal Itemsets?

A database with m distinct items has 2^m possible subsets. Even a modest database with 100 items has more possible subsets than atoms in the universe. While most of these are not frequent, the number of frequent itemsets can still be enormous—potentially millions or more.

Maximal frequent itemsets solve this information overload problem. A frequent itemset is maximal if no proper superset of it is also frequent. The key insight is that every frequent itemset is a subset of at least one maximal frequent itemset, so the maximal set is a **lossless compression** of the full set of frequent itemsets.

1.4 Project Objectives

This project has five objectives:

1. Formally define the problem of mining frequent maximal itemsets from uncertain databases using both the expected support model and the probabilistic support model.
2. Design and implement four algorithms: UGenMax and UFPMax (expected support), TODIS-MAX and APFI-MAX (probabilistic support).
3. Implement all algorithms as a pure Java SE application with no external library dependencies, with separate algorithm classes and experiment classes.
4. Experimentally evaluate and compare all four algorithms on benchmark datasets (mushroom, retail, accidents) of varying sizes and uncertainty levels.
5. Analyze the trade-offs between exact and approximate probabilistic mining.

1.5 Report Structure

The remainder of this report is organized as follows. Chapter 2 reviews related work. Chapter 3 provides formal problem definitions for both support models. Chapter 4 presents all four algorithms in detail. Chapter 5 describes the experimental setup. Chapter 6 presents results. Chapter 7 concludes.

CHAPTER 2
LITERATURE REVIEW

2.1 Frequent Itemset Mining in Deterministic Databases

The Apriori algorithm [2] was the first efficient algorithm for frequent itemset mining. Its key contribution was the **anti-monotonicity property**: if an itemset is infrequent, then all of its supersets are also infrequent. The FP-Growth algorithm [9] improved upon Apriori by eliminating candidate generation through a compressed FP-tree data structure.

2.2 Mining Maximal Frequent Itemsets

Several algorithms have been developed specifically for maximal frequent itemset mining:

- **MaxMiner** [4] was the first to use a look-ahead pruning strategy.
- **GenMax** [7] introduced depth-first backtracking with tidsets and progressive focusing. Our UGenMax adapts this algorithm.
- **FPMax** [8] extended FP-Growth with MFI-tree superset checking. Our UFPMax adapts this algorithm.
- **CARPENTER-MAX** [13] was designed for datasets with many items but few transactions.
- **Charm-MFI** [16] discovers maximal itemsets by post-processing closed itemsets.

2.3 Uncertain Database Mining

Chui, Kao, and Hung [5] pioneered uncertain data mining with the expected support model and U-Apriori algorithm. Leung et al. [10] proposed UF-Growth for uncertain data. Leung and Tanbeer [11] improved this with UFP-Growth. Li and Zhang [12] addressed the top-k variant with dynamic threshold raising. Aggarwal et al. [3] analyzed the theoretical complexity of uncertain pattern mining.

2.4 Probabilistic Support Mining

Sun et al. [14] introduced the **probabilistic support model**, which defines frequency as $\Pr[\text{sup}(X) \geq T] \geq \tau$, giving a statistically rigorous definition of frequency under uncertainty. Their TODIS framework uses exact PMF computation via Dynamic Programming. Chen et al. [15] proposed

APFI-MAX, which approximates the PMF using the Central Limit Theorem for faster computation, trading small accuracy loss for significant speed improvement ($O(n)$ vs. $O(n \cdot T)$).

2.5 Gap and Our Contribution

Most prior work focuses on mining **all** frequent or **closed** frequent itemsets from uncertain data. Mining **maximal** frequent itemsets has received less attention. Furthermore, few works provide a unified comparison across both expected support and probabilistic support models. Our contribution fills these gaps by: (1) adapting GenMax and FPMax for uncertain data with expected support, (2) implementing TODIS-MAX and APFI-MAX for probabilistic maximal mining, and (3) providing comprehensive experimental comparison across all four approaches.

CHAPTER 3
PROBLEM STATEMENT AND RELATED DEFINITIONS

This section provides precise mathematical definitions for each concept. We build the definitions incrementally, starting from the data model and working up to the problem statement.

3.1 Uncertain Transaction Database

Definition 3.1 (Item Set)**.** *Let $I = \{i_1, i_2, \ldots, i_m\}$ be a finite set of m distinct items.*

Definition 3.2 (Uncertain Transaction)**.** *An uncertain transaction t_k is a set of items from I, where each item i is associated with an* ***existential probability*** *$P(i, t_k)$ satisfying:*

$$0 < P(i, t_k) \leq 1$$

Items are assumed to be mutually independent within each transaction.

Definition 3.3 (Uncertain Transaction Database)**.** *An uncertain transaction database $D = \{t_1, t_2, \ldots, t_n\}$ is an ordered collection of n uncertain transactions.*

Running Example. Throughout this report, we use the following 5-transaction database for illustration:

Table 3.1: The contextUncertain dataset

Transaction	**Items (with existential probabilities)**			
t_1	1(0.8)	3(0.6)	4(0.9)	
t_2	2(0.7)	3(0.5)	5(0.9)	
t_3	1(0.6)	2(0.8)	3(0.7)	5(0.85)
t_4	2(0.5)	5(0.6)		
t_5	1(0.9)	2(0.75)	3(0.8)	5(0.7)

3.2 Expected Support

Definition 3.4 (Expected Support)**.** *The expected support of an itemset $X = \{x_1, x_2, \ldots, x_p\}$ in database D is:*

$$\textit{expSup}(X) = \sum_{t \in D} \prod_{x \in X} P(x, t) \tag{3.1}$$

Worked Example. Consider $X = \{1, 3\}$:

$$
\begin{aligned}
&t_1 : P(1, t_1) \times P(3, t_1) = 0.8 \times 0.6 = 0.48 \\
&t_2 : \text{item 1 absent} = 0.00 \\
&t_3 : P(1, t_3) \times P(3, t_3) = 0.6 \times 0.7 = 0.42 \\
&t_4 : \text{item 1 absent} = 0.00 \\
&t_5 : P(1, t_5) \times P(3, t_5) = 0.9 \times 0.8 = 0.72
\end{aligned}
$$

$$\text{expSup}(\{1, 3\}) = 0.48 + 0.00 + 0.42 + 0.00 + 0.72 = \mathbf{1.62}$$

3.3 Anti-Monotonicity Property

Theorem 3.1 (Anti-Monotonicity of Expected Support)**.** *For any itemset X and any item $y \notin X$:*

$$\textit{expSup}(X \cup \{y\}) \leq \textit{expSup}(X)$$

Chứng minh. Since $0 < P(y, t) \leq 1$ for all transactions t, each term $\prod_{x \in X} P(x, t) \cdot P(y, t) \leq \prod_{x \in X} P(x, t)$. Therefore the sum cannot increase. □

3.4 Frequent and Maximal Frequent Itemsets

Definition 3.5 (Frequent Itemset)**.** *Given threshold minsup, itemset X is **frequent** if expSup$(X) \geq$ minsup.*

Definition 3.6 (Maximal Frequent Itemset)**.** *A frequent itemset X is **maximal** if for every item $y \notin X$, expSup$(X \cup \{y\}) <$ minsup.*

3.5 Probabilistic Support

While expected support provides a single aggregated value, the **probabilistic support model** captures the full probability distribution of an itemset's support count.

Definition 3.7 (Support Random Variable)**.** *Under the Possible World Semantics, the support of itemset X in database D is a random variable:*

$$\textit{sup}(X) = \sum_{t \in D} \mathbf{1}[X \subseteq t] \tag{3.2}$$

*where $\mathbf{1}[X \subseteq t]$ is a Bernoulli random variable with success probability $p_t^X = \prod_{x \in X} P(x, t)$. Since transactions are independent, sup(X) follows a **Poisson Binomial distribution** with parameters $(p_1^X, p_2^X, \ldots, p_n^X)$.*

Definition 3.8 (Probabilistic Frequent Itemset (p-FP))**.** *Given integer threshold T (minsup) and probability threshold τ (minprob), itemset X is a* ***probabilistic frequent itemset*** *(p-FP) if:*

$$\Pr[sup(X) \geq T] \geq \tau \tag{3.3}$$

Definition 3.9 (Maximal Probabilistic Frequent Itemset)**.** *A p-FP X is* ***maximal*** *if no proper superset of X is also a p-FP.*

The expected support relates to the probabilistic support: $E[\text{sup}(X)] = \sum_t p_t^X = \text{expSup}(X)$. The variance is: $\text{Var}[\text{sup}(X)] = \sum_t p_t^X(1 - p_t^X)$.

3.6 Problem Statement

This project addresses two variants of the problem:

Problem 1 (Expected Support Model):

- **Input:** Uncertain database D, threshold minsup > 0 (or integer K for Top-K mode).
- **Output:** All maximal frequent itemsets where $\text{expSup}(X) \geq$ minsup.
- **Algorithms:** UGenMax, UFPMax.

Problem 2 (Probabilistic Support Model):

- **Input:** Uncertain database D, integer threshold T, probability threshold $\tau \in (0, 1]$.
- **Output:** All maximal p-FPs where $\Pr[\text{sup}(X) \geq T] \geq \tau$.
- **Algorithms:** TODIS-MAX (exact), APFI-MAX (approximate).

CHAPTER 4
METHODS

This chapter presents both algorithms in detail: data structures, pseudocode, worked examples, and complexity analysis.

4.1 Branch-and-Bound Framework

Both algorithms explore the **itemset lattice** using depth-first search (DFS). The search is a tree: the root is the empty set, each node is an itemset (prefix), and each edge adds one item. Without pruning, this requires exploring 2^m subsets. Branch-and-bound makes the search tractable by computing **bounds** at each node and **pruning** subtrees that cannot yield useful results.

4.1.1 Pruning 1: Anti-Monotonicity Bound

Rule: If $\text{expSup}(\text{prefix} \cup \{\text{item}\}) < \text{minsup}$, skip this item and the entire subtree below it.

Rationale: By Theorem 3.1, every superset will have even lower expected support. This rule is responsible for the majority of pruning in practice. For UGenMax, a cheaper upper bound $\min(\text{expSup}(A), \text{expSup}(B))$ is checked before the full intersection.

4.1.2 Pruning 2: Progressive Focusing

Rule: Let P be the current prefix and C the remaining candidates. If $P \cup C \subseteq M$ for some known MFI M, prune the entire subtree.

Rationale: Any discoverable itemset would be a subset of M and therefore not maximal. This rule becomes increasingly effective as more MFIs are discovered during the search.

4.1.3 Pruning 3: Top-K Dynamic Threshold Raising

In Top-K mode, a **min-heap** of size K tracks the best results. When a new MFI is discovered, if the heap is full and the new MFI's expected support exceeds the heap minimum, the minimum is replaced. The new heap minimum becomes the updated minsup.

Feedback loop: Better results $\rightarrow$ higher threshold $\rightarrow$ more aggressive pruning $\rightarrow$ faster convergence.

Design difference: UGenMax raises the threshold dynamically during search. UFPMax does **not**, because its processing order discovers single-item MFIs early, which would prematurely block multi-item MFIs.

4.2 Algorithm 1: UGenMax

UGenMax adapts GenMax [7] for uncertain data using **bottom-up, depth-first backtracking** with **weighted tidsets**.

4.2.1 Data Structure: Weighted Tidsets

Definition 4.1 (Weighted Tidset)**.** *The weighted tidset of item i is:*

$$WT(i) = \{(t, P(i,t)) \mid t \in D,\ P(i,t) > 0\}$$

sorted by transaction ID. Expected support: $expSup(i) = \sum_{(t,p)\in WT(i)} p$.

To compute expSup($\{A, B\}$), we **intersect** WT(A) and WT(B) using a merge-join in $O(|\text{WT}(A)| + |\text{WT}(B)|)$ time. For each common transaction t, the result is $(t, P(A,t) \times P(B,t))$.

Worked Example. Using items 1 and 3 from Table 3.1:

$$\text{WT}(1) = [(t_1, 0.8),\ (t_3, 0.6),\ (t_5, 0.9)] \quad \text{expSup} = 2.3$$
$$\text{WT}(3) = [(t_1, 0.6),\ (t_2, 0.5),\ (t_3, 0.7),\ (t_5, 0.8)] \quad \text{expSup} = 2.6$$

Intersection:

$$\begin{aligned} &t_1 : \text{both present} \rightarrow 0.8 \times 0.6 = 0.48 \\ &t_2 : \text{only in WT}(3) \rightarrow \text{skip} \\ &t_3 : \text{both present} \rightarrow 0.6 \times 0.7 = 0.42 \\ &t_5 : \text{both present} \rightarrow 0.9 \times 0.8 = 0.72 \end{aligned}$$

$$\text{WT}(\{1,3\}) = [(t_1, 0.48),\ (t_3, 0.42),\ (t_5, 0.72)] \quad \text{expSup} = 1.62$$

4.2.2 Enumeration Strategy

Items are sorted by expected support **ascending**. Candidates are processed **most frequent first** (right to left). This ordering discovers large MFIs early, strengthening progressive focusing.

4.2.3 Pseudocode

Algorithm 1 UGenMax

Require: Uncertain database D, threshold minsup
Ensure: Set MFI of all maximal frequent itemsets

```
MFI ← ∅
for each item i in D do
    Build WT(i) by scanning all transactions
end for
Remove items where expSup(i) < minsup
Sort items by expSup ascending: [i_1, i_2, ..., i_k]
GenMax(∅, [i_1, ..., i_k], null)
return MFI
```

Algorithm 2 GenMax Procedure

```
procedure GenMax(prefix, candidates, prefixTidset)
    if prefix ∪ candidates ⊆ some MFI in MFI then                ▷ Pruning 2
        return
    end if
    hasFreqExt ← false
    for idx ← |candidates| − 1 downto 0 do
        item ← candidates[idx]
        newTidset ← Intersect(prefixTidset, WT(item))
        expSup ← Sum(newTidset.probabilities)
        if expSup < minsup then                                  ▷ Pruning 1
            continue
        end if
        hasFreqExt ← true
        newPrefix ← prefix ∪ {item}
        newCands ← {c ∈ candidates[0..idx−1] | Intersect(newTidset, WT(c)).expSup ≥
    minsup}
        if newCands = ∅ then
            AddMaximal(newPrefix, expSup)
        else
            GenMax(newPrefix, newCands, newTidset)
        end if
    end for
    if not hasFreqExt and prefix ≠ ∅ then
        AddMaximal(prefix, prefixTidset.expSup)
    end if
end procedure
```

4.2.4 Complexity Analysis

Time: Worst case $O(2^m)$ nodes, each with $O(n)$ tidset intersection. Pruning dramatically reduces explored nodes in practice.

Space: $O(m \times n)$ for weighted tidsets, $O(|\text{MFI}|)$ for results, $O(m)$ for the recursion stack.

4.3 Algorithm 2: UFPMax

UFPMax adapts FPMax [8] for uncertain data using **top-down, recursive conditional database projection**.

4.3.1 Data Structure: Projected Transactions

Each projected transaction is a triple (weight, items[], probs[]):

- **weight**: accumulated product of probabilities of all prefix items. Initially 1.0.
- **items[] and probs[]**: remaining items and probabilities, sorted by frequency-descending order.

Expected support of item i in a conditional database:

$$\text{expSup}(i \mid \text{prefix}) = \sum_{t:i \in t} \text{weight}_t \times P(i,t) \tag{4.1}$$

4.3.2 Conditional Database Construction

When extending prefix by item i:

1. Select transactions containing item i.
2. Compute new_weight = old_weight $\times P(i,t)$.
3. Project to items **before** i in header order (more frequent items).
4. Remove items with conditional expSup $<$ minsup.

4.3.3 Pseudocode

Algorithm 3 UFPMax

Require: Uncertain database D, threshold minsup
Ensure: Set MFI of all maximal frequent itemsets

```
MFI ← ∅
Compute expSup for each single item
Remove items with expSup < minsup
Sort items by expSup descending: header = [h_1, ..., h_k]
Build initial projected DB (weight = 1.0, frequent items only)
FPMAX(∅, 0.0, projDB, header)
return MFI
```

Algorithm 4 FPMax Procedure

```
procedure FPMAX(prefix, prefixExpSup, condDB, headerItems)
    if prefix ∪ headerItems ⊆ some MFI then                    ▷ Pruning 2
        return
    end if
    hasFreqExt ← false
    for idx ← |headerItems| − 1 downto 0 do
        item ← headerItems[idx]
        expSup ← Σ_{t∈condDB} weight_t × P(item, t)
        if expSup < minsup then                                ▷ Pruning 1
            continue
        end if
        hasFreqExt ← true; newPrefix ← prefix ∪ {item}
        newCondDB ← BUILDCONDITIONAL(condDB, item, idx)
        newHeader ← {h ∈ header[0..idx−1] | condExpSup(h) ≥ minsup}
        if newHeader = ∅ then
            ADDMAXIMAL(newPrefix, expSup)
        else
            FPMAX(newPrefix, expSup, newCondDB, newHeader)
        end if
    end for
    if not hasFreqExt and prefix ≠ ∅ then
        ADDMAXIMAL(prefix, prefixExpSup)
    end if
end procedure
```

4.3.4 Complexity Analysis

Time: Worst case $O(2^m)$ nodes. Conditional DB construction is $O(|\text{condDB}| \times |\text{items}|)$ per node, shrinking at deeper levels.

Space: $O(n \times m)$ for the initial projected DB. Each recursive level maintains its own conditional DB. Stack depth at most m.

4.4 Comparison of UGenMax and UFPMax

Table 4.1: Comparison of UGenMax and UFPMax

Aspect	UGenMax	UFPMax
Base algorithm	GenMax (Gouda & Zaki, 2005)	FPMax (Grahne & Zhu, 2003)
Approach	Bottom-up (vertical)	Top-down (horizontal)
Data structure	Weighted tidsets	Projected conditional DBs
Item ordering	Ascending expSup	Descending expSup
Top-K threshold	Raised dynamically	Fixed, select after search
Best for	Sparse data	Dense data

4.5 Algorithm 3: TODIS-MAX

TODIS-MAX (Top-Down Inheritance of Support for Maximal itemsets) uses the **probabilistic support model** to mine maximal p-FPs with exact probability computation. It is based on the framework of Sun et al. [14].

4.5.1 Probabilistic Support Computation

The key challenge is computing $\Pr[\text{sup}(X) \geq T]$ for a given itemset X. Since sup(X) follows a Poisson Binomial distribution, the probability mass function (PMF) can be computed via **Dynamic Programming**:

Let $f_k(j)$ = probability that j of the first k transactions contain X. Then:

$$f_0(0) = 1 \tag{4.2}$$

$$f_k(j) = f_{k-1}(j) \cdot (1 - p_k^X) + f_{k-1}(j-1) \cdot p_k^X \tag{4.3}$$

This requires $O(n \cdot T)$ time and $O(T)$ space. Then $\Pr[\text{sup}(X) \geq T] = \sum_{j=T}^{n} f_n(j)$.

4.5.2 Algorithm Overview

Phase 1 — Apriori-based Candidate Generation:

1. Start with single items; apply level-wise join and pruning.
2. **Pruning by count:** If cnt$(X) < T$, prune (Lemma 2 from [14]). Here cnt(X) is the number of transactions containing all items in X.
3. **Pruning by Chernoff bound:** If $E[\text{sup}(X)]$ is too low for $\Pr[\text{sup}(X) \geq T] \geq \tau$ to hold, prune without computing the full PMF.

Phase 2 — Top-Down Maximal Verification:

1. Sort candidates by length descending.
2. For each candidate (longest first), skip if it is a subset of any confirmed maximal p-FP.
3. Compute exact PMF via DP and check $\Pr[\text{sup}(X) \geq T] \geq \tau$.
4. Confirmed candidates become maximal p-FPs.

4.5.3 Pseudocode

Algorithm 5 TODIS-MAX

Require: Uncertain database D, integer threshold T, probability threshold τ
Ensure: Set of all maximal p-FPs

```
C_1 ← single items passing count and Chernoff pruning
k ← 1
while C_k ≠ ∅ do
    C_{k+1} ← Apriori-join(C_k) with count + Chernoff pruning
    k ← k + 1
end while
Sort all candidates by length descending
MFI ← ∅
for each candidate X (longest first) do
    if X ⊆ some M ∈ MFI then
        skip                                        ▷ Subset of known maximal
    end if
    Compute PMF via DP, get Pr[sup(X) ≥ T]
    if Pr[sup(X) ≥ T] ≥ τ then
        MFI ← MFI ∪ {X}
    end if
end for
return MFI
```

4.5.4 Complexity Analysis

Time: Candidate generation is $O(\sum |C_k|^2)$. PMF computation is $O(n \cdot T)$ per candidate. Phase 2 processes at most $\sum |C_k|$ candidates.

Space: $O(T)$ for the DP array, $O(\sum |C_k|)$ for storing candidates.

4.6 Algorithm 4: APFI-MAX

APFI-MAX (Approximation of Probabilistic Frequent Itemset-MAX) is a faster alternative to TODIS-MAX that uses the **Central Limit Theorem (CLT)** for approximate probability estimation. It is based on Chen et al. [15].

4.6.1 CLT Approximation

For large n, the Poisson Binomial $\text{sup}(X)$ is approximately Normal:

$$\text{sup}(X) \sim \mathcal{N}\left(\mu, \sigma^2\right), \quad \mu = \sum_t p_t^X, \quad \sigma^2 = \sum_t p_t^X (1 - p_t^X) \tag{4.4}$$

The frequency is estimated as:

$$\Pr[\text{sup}(X) \geq T] \approx 1 - \Phi\left(\frac{T - \mu}{\sigma}\right) \tag{4.5}$$

where Φ is the standard normal CDF. This runs in $O(n)$ per itemset — significantly faster than the $O(n \cdot T)$ exact DP.

4.6.2 Expectation Bounds

APFI-MAX uses tighter pruning via expectation bounds derived from the CLT:

- **Lower bound:** If $E[\text{sup}(X)] < lb(E)$, then X cannot be a p-FP regardless of variance. The lower bound $lb(E)$ is computed from the Chernoff inequality.
- **Upper bound:** If $E[\text{sup}(X)] \geq ub(E)$, then X is definitely a p-FP. The upper bound is $ub(E) = T + z_\tau \cdot \sqrt{E}$, where $z_\tau = \Phi^{-1}(1 - \tau)$.

4.6.3 Algorithm Overview

Step 1 — CGEB (Candidate Generation with Expectation Bound): Apriori-based generation with two criteria per candidate: (a) $\text{cnt}(X) \geq T$, and (b) $E[\text{sup}(X)] \geq lb(E)$.

Step 2 — Top-Down Confirmation: Process candidates from longest to shortest. If a candidate is a subset of a confirmed p-FP from a longer level, it inherits frequent status (skip the check). Otherwise, apply FM.

Step 3 — FM (Frequency Measurement): If $E[\text{sup}(X)] \geq ub(E)$, automatically frequent. Otherwise, estimate via CLT (Equation 4.5).

4.6.4 Pseudocode

Algorithm 6 APFI-MAX

Require: Uncertain database D, integer threshold T, probability threshold τ
Ensure: Set of maximal p-FPs (approximate)

```
Compute lb(E) and ub(E) from T and τ
C_1 ← items with cnt ≥ T and E ≥ lb(E)
Generate higher-level candidates via Apriori with expectation bound
Sort all candidates by length descending
MFI ← ∅; Fre_Pre ← ∅
for each candidate X (longest first) do
    if X ⊆ some Y ∈ Fre_Pre then
        Mark X as frequent (inherited)                    ▷ Skip verification
        continue
    end if
    if E(X) ≥ ub(E) then
        Mark X as frequent                                ▷ Upper bound
    else if 1 − Φ( (T − E(X)) / √Var(X) ) ≥ τ then
        Mark X as frequent                                ▷ CLT
    end if
    if X is frequent and X ⊈ any M ∈ MFI then
        MFI ← MFI ∪ {X}
    end if
end for
return MFI
```

4.6.5 Complexity Analysis

Time: Same as TODIS-MAX for candidate generation. Frequency measurement is $O(n)$ per candidate (vs. $O(n \cdot T)$ for exact DP), giving a significant speedup on large datasets.

Space: $O(\sum |C_k|)$ for candidates plus $O(1)$ per frequency check (no DP array needed).

4.7 Comparison of All Four Algorithms

4.8 Top-K Seeding

Both UGenMax and UFPMax support a Top-K seeding phase: (1) take the top-50 most frequent items, (2) generate all single and 2-item combinations, (3) fill a Top-K heap, (4) set initial minsup to 50% of the heap minimum, (5) reset the heap and begin the main search. If fewer than K results are found, retry with 25% of current threshold, up to 3 retries.

Table 4.2: Comparison of all four algorithms

Aspect	**UGenMax / UFPMax**	**TODIS-MAX / APFI-MAX**
Support model	Expected support	Probabilistic support
Threshold	expSup$(X) \geq$ minsup	$\Pr[\text{sup}(X) \geq T] \geq \tau$
Search strategy	Branch-and-bound DFS	Apriori + top-down verification
Pruning	Anti-monotonicity, progressive focusing	Count, Chernoff/expectation bound
Prob. computation	Not needed	DP (exact) / CLT (approx.)
Top-K mode	Yes	No

4.9 Maximality Checking

When a leaf node is reached, the candidate MFI is checked: (1) it must not be a subset of any existing MFI, and (2) existing MFIs that are subsets of the new candidate are removed. Both checks use sorted arrays for $O(|A| + |B|)$ merge-based comparison.

CHAPTER 5
EXPERIMENT DESIGN

5.1 Implementation Details

All four algorithms are implemented in Java SE 25 with no external dependencies. The code is organized into packages:

- `algorithm/`: UGenMax, UFPMax, TODISMAX, APFIMAX, SupportPMF, WeightedTidset
- `model/`: UncertainDatabase, UncertainTransaction, Itemset, TopKHeap
- `util/`: PerformanceTracker
- `experiment/`: ExperimentRunner, ExpUFPMax, ExpUGenMax, ExpTODISMAX, ExpAPFIMAX

Algorithm classes contain only the algorithm logic (no `main` method). Experiment classes contain `main` methods that load the database, run the algorithm, and output results with performance statistics. Every class includes Javadoc documentation.

5.2 Datasets

Table 5.1: Experimental datasets

Dataset	Trans.	Items	Density	Source
contextUncertain	5	5	Dense	Manual
mushroom	8,416	120	Dense	UCI
retail	88,162	16,470	Sparse	SPMF
accidents	340,183	468	Very dense	SPMF

Each standard dataset is converted to uncertain format at three uncertainty levels using the `DatasetConverter` utility:

Table 5.2: Uncertainty level distributions

Level	Distribution	Simulates
High certainty	$N(\mu{=}0.8, \sigma{=}0.1)$, clamped to $[0.1, 1.0]$	Low uncertainty
Medium certainty	$N(\mu{=}0.5, \sigma{=}0.2)$, clamped to $[0.1, 1.0]$	Moderate uncertainty
Low certainty	$N(\mu{=}0.3, \sigma{=}0.15)$, clamped to $[0.1, 1.0]$	High uncertainty

This methodology follows standard practice in uncertain data mining research [5, 10].

5.3 Configurations

Expected support model (UGenMax, UFPMax): Vary minsup across appropriate ranges per dataset. Top-K mode with $K \in \{5, 10, 20, 50\}$.

Probabilistic support model (TODIS-MAX, APFI-MAX): Vary integer minsup and $\tau \in \{0.5, 0.7, 0.9\}$.

For each configuration we record: time (ms), memory (MB), number of MFIs, and algorithm-specific statistics (nodes explored, pruning counts, candidates generated).

5.4 Correctness Validation

- UGenMax and UFPMax produce identical MFI sets for all configurations (cross-algorithm agreement).
- Hand verification on contextUncertain for all four algorithms.
- TODIS-MAX and APFI-MAX compared: APFI-MAX may produce slightly different results due to CLT approximation, but should closely match TODIS-MAX.

5.5 Performance Measurement

PerformanceTracker uses `System.nanoTime()` for timing and `Runtime.totalMemory() - Runtime.freeMemory()` for memory, with GC forced before each run. Each expected support experiment is averaged over 3 measured runs after 1 warm-up run.

CHAPTER 6
EXPERIMENT RESULTS

6.1 Correctness Verification

On contextUncertain, all four algorithms produce correct results verified by hand computation. For expected support algorithms, UGenMax and UFPMax produce **identical MFI sets** across all datasets and all minsup values tested, confirming correctness. For probabilistic algorithms, TODIS-MAX and APFI-MAX produce identical results on mushroom, confirming that the CLT approximation is accurate for this dataset.

6.2 Expected Support: Effect of Minimum Support Threshold

Table 6.1 shows the performance of UFPMax and UGenMax on mushroom_medium with varying minsup thresholds.

Table 6.1: Performance on mushroom_medium (8,416 transactions, 120 items)

minsup	**MFIs**	**Time (ms)**		**Memory (MB)**	
		UFPMax	UGenMax	UFPMax	UGenMax
500	314	920	417	74.4	65.5
1000	73	871	517	16.1	45.7
1500	30	285	159	43.2	21.3
2000	14	191	147	4.6	23.0
2500	9	157	130	13.8	18.5
3000	5	170	70	3.6	14.0

Observations:

- As minsup increases, both the number of MFIs and execution time decrease, as expected from anti-monotonicity pruning.
- UGenMax is consistently faster than UFPMax on this dense dataset, especially at higher minsup values (e.g., 70 ms vs. 170 ms at minsup=3000).
- At low minsup (500), the search space is large (314 MFIs) and both algorithms require more time.

Table 6.2 shows results on the retail dataset, which is much sparser.

Observations:

- Retail produces fewer MFIs than mushroom due to its sparse nature (average transaction length ≈ 10 items out of 16,470 distinct items).

Table 6.2: Performance on retail_medium (88,162 transactions, 16,470 items)

minsup	**MFIs**	**Time (ms)**		**Memory (MB)**	
		UFPMax	UGenMax	UFPMax	UGenMax
1000	18	736	344	132.0	78.0
2000	6	611	351	126.0	78.0
3000	4	609	249	125.5	73.1
5000	4	496	399	124.0	72.0
8000	2	386	261	110.5	69.5
10000	2	410	251	111.0	68.5

- UGenMax is faster than UFPMax here as well, with roughly half the memory usage.
- The time is dominated by database loading and tidset/projection construction rather than the search itself.

6.3 Scalability: Accidents Dataset

Table 6.3 shows results on the accidents dataset, which is the largest and densest.

Table 6.3: Performance on accidents_medium (340,183 transactions, 468 items)

minsup	**MFIs**	**Time (s)**		**Memory (MB)**	
		UFPMax	UGenMax	UFPMax	UGenMax
30,000	453	45.9	38.7	2,026	995
50,000	154	15.8	13.3	2,102	1,637
70,000	48	7.9	5.8	1,625	2,092
100,000	21	5.3	3.2	827	1,124

Observations:

- Both algorithms scale to 340K transactions with results in seconds to tens of seconds.
- UGenMax is 15–40% faster than UFPMax across all minsup values on this very dense dataset.
- At low minsup (30,000), the search discovers 453 MFIs, demonstrating that the algorithms handle large result sets effectively.
- Memory usage reaches 2 GB at low thresholds due to the large number of items per transaction ($\sim$33 items on average out of 468).

6.4 Effect of Uncertainty Level

Table 6.4 shows how uncertainty level affects results on mushroom with fixed minsup=1500.

Table 6.4: Effect of uncertainty level on mushroom (minsup=1500)

Uncertainty	**MFIs**	**Time (ms)**		**Memory (MB)**	
		UFPMax	UGenMax	UFPMax	UGenMax
High (μ=0.8)	341	701	415	5.2	74.4
Medium (μ=0.5)	30	268	151	43.7	21.5
Low (μ=0.3)	9	264	87	11.0	16.5

Key finding: Higher certainty (high level) produces dramatically more MFIs (341 vs. 9 for low), because expected supports are higher when item probabilities are near 1.0. This means low-uncertainty databases require lower minsup thresholds to find meaningful patterns. Execution time also increases with more MFIs due to larger search space.

6.5 Probabilistic Support: TODIS-MAX vs. APFI-MAX

Table 6.5 compares the two probabilistic algorithms on mushroom_medium.

Table 6.5: TODIS-MAX vs. APFI-MAX on mushroom_medium

minsup	τ	**MFIs**		**Time (ms)**	
		TODIS	APFI	TODIS	APFI
3000	0.5	5	5	996	226
3000	0.7	5	5	1702	224
3000	0.9	5	5	1373	203
4000	0.5	3	3	737	240
4000	0.7	3	3	656	191
4000	0.9	3	3	660	192

Key findings:

- APFI-MAX is **3–8$\times$ faster** than TODIS-MAX, confirming that the $O(n)$ CLT approximation is significantly faster than the $O(n \cdot T)$ exact DP computation.
- Both produce **identical results** on this dataset, indicating that the CLT approximation is accurate for the mushroom dataset size ($n = 8416$).
- Both algorithms generate the same number of candidates (135 at minsup=3000), as they use similar Apriori-based candidate generation.
- Higher minprob (τ) does not reduce the result count here because the p-FPs have high probability of being frequent.
- APFI-MAX uses significantly less memory ($\sim$18 MB vs. up to 180 MB for TODIS-MAX) since it avoids the DP array.

6.6 Cross-Model Comparison

The expected support model (UFPMax/UGenMax) and probabilistic support model (TODIS-MAX/APFI-MAX) serve complementary purposes:

- **Expected support** is faster and simpler but treats all transactions equally: an itemset with 0.01 probability in each of 1000 transactions can have the same expected support as one with 1.0 probability in 10 transactions.
- **Probabilistic support** distinguishes these cases by asking "what is the probability that this itemset appears at least T times?" This gives a more statistically rigorous measure of frequency.
- For practitioners: use expected support for quick exploratory analysis, and probabilistic support when statistical guarantees matter.

6.7 Summary of Findings

1. **Correctness:** UGenMax and UFPMax produce identical results. TODIS-MAX and APFI-MAX produce matching results.
2. **UGenMax vs. UFPMax:** UGenMax is consistently faster (15–50%) with lower memory usage on all tested datasets (dense mushroom, sparse retail, and very dense accidents).
3. **TODIS-MAX vs. APFI-MAX:** APFI-MAX is 3–8$\times$ faster with much less memory, while producing identical results on the tested datasets.
4. **Uncertainty impact:** Higher certainty produces more MFIs; lower certainty requires lower thresholds.
5. **Scalability:** All algorithms scale to 340K transactions (accidents) within acceptable time limits.

CHAPTER 7
CONCLUSIONS

7.1 Summary

This project designed, implemented, and evaluated four algorithms for mining frequent maximal itemsets from uncertain transaction databases, covering two support models:

- **Expected support model:** UGenMax (bottom-up weighted tidset intersections, adapted from GenMax) and UFPMax (top-down conditional database projection, adapted from FPMax). Both use branch-and-bound with three pruning strategies: anti-monotonicity, progressive focusing, and dynamic Top-K threshold raising.
- **Probabilistic support model:** TODIS-MAX (exact PMF computation via Dynamic Programming, based on Sun et al., 2010) and APFI-MAX (approximate frequency estimation via CLT, based on Chen et al., 2020). Both use Apriori-based candidate generation with top-down maximal verification.

Experiments on three benchmark datasets (mushroom, retail, accidents) at three uncertainty levels demonstrate that all algorithms produce correct results and exhibit distinct performance characteristics.

7.2 Key Contributions

1. Formalized the problem with precise mathematical definitions for both expected support and probabilistic support models.
2. Adapted GenMax and FPMax for uncertain data with expected support, including Top-K mode.
3. Implemented TODIS-MAX and APFI-MAX for probabilistic maximal frequent itemset mining.
4. Clean Java SE implementation with no external dependencies, organized into algorithm classes (no main) and experiment classes (with main and statistics output).
5. Comprehensive experimental comparison across 3 datasets, 3 uncertainty levels, 4 algorithms, and varying parameter configurations, revealing that UGenMax outperforms UFPMax and APFI-MAX outperforms TODIS-MAX in speed.

7.3 Limitations

- Maximality checking uses linear scan ($O(|\text{MFI}|)$ per check); a trie or hash-based approach could improve this.
- APFI-MAX's CLT approximation may be less accurate for small datasets where the normal approximation is poor.
- TODIS-MAX and APFI-MAX do not support Top-K mode.
- Memory usage on the accidents dataset reaches 2 GB at low thresholds.

7.4 Future Work

Possible extensions include:

- Diffset optimization for UGenMax to reduce memory usage.
- Parallel search across independent subtrees for multi-core speedup.
- Extension to mining closed or sequential patterns from uncertain data.
- Evaluation on real-world uncertain datasets from sensor networks or noisy data integration.
- Top-K mode for probabilistic algorithms.
- GPU acceleration for the DP-based PMF computation in TODIS-MAX.

Bibliography

[1] Agrawal, R., Imielinski, T., and Swami, A. (1993). Mining association rules between sets of items in large databases. *Proc. ACM SIGMOD*, pp. 207–216.

[2] Agrawal, R. and Srikant, R. (1994). Fast algorithms for mining association rules. *Proc. VLDB*, pp. 487–499.

[3] Aggarwal, C.C., Li, Y., Wang, J., and Wang, J. (2009). Frequent pattern mining with uncertain data. *Proc. ACM SIGKDD*, pp. 29–38.

[4] Bayardo, R.J. (1998). Efficiently mining long patterns from databases. *Proc. ACM SIGMOD*, pp. 85–93.

[5] Chui, C.K., Kao, B., and Hung, E. (2007). Mining frequent itemsets from uncertain data. *Proc. PAKDD*, pp. 47–58.

[6] Fournier-Viger, P., et al. (2014). SPMF: A Java open-source pattern mining library. *JMLR*, 15, pp. 3569–3573.

[7] Gouda, K. and Zaki, M.J. (2005). GenMax: An efficient algorithm for mining maximal frequent itemsets. *Data Mining and Knowledge Discovery*, 11(3), pp. 223–242.

[8] Grahne, G. and Zhu, J. (2003). Efficiently using prefix-trees in mining frequent itemsets. *Proc. IEEE ICDM Workshop on FIMI*.

[9] Han, J., Pei, J., and Yin, Y. (2000). Mining frequent patterns without candidate generation. *Proc. ACM SIGMOD*, pp. 1–12.

[10] Leung, C.K.-S., Mateo, M.A.F., and Brajczuk, D.A. (2008). A tree-based approach for frequent pattern mining from uncertain data. *Proc. PAKDD*, pp. 653–661.

[11] Leung, C.K.-S. and Tanbeer, S.K. (2013). Fast tree-based mining of frequent itemsets from uncertain data. *Proc. DASFAA*, pp. 272–287.

[12] Li, H. and Zhang, N. (2019). Mining top-k frequent itemsets from uncertain databases. *Proc. DSAA*.

[13] Pan, F., Cong, G., Tung, A.K.H., Yang, J., and Zaki, M.J. (2003). CARPENTER: Finding closed patterns in long biological datasets. *Proc. ACM SIGKDD*, pp. 637–642.

[14] Sun, L., Cheng, R., Cheung, D.W., and Cheng, J. (2010). Mining uncertain data with probabilistic guarantees. *Proc. ACM SIGKDD*, pp. 273–282.

[15] Chen, L., Liu, Y., Zheng, Z., and Yu, P.S. (2020). Approximation of probabilistic maximal frequent itemset mining over uncertain sensed data. *IEEE Access*, 8, pp. 51513–51526.

[16] Szathmary, L., Napoli, A., and Valtchev, P. (2006). Towards rare itemset mining. *Proc. IEEE ICTAI*.